

BACHELOR THESIS

Design and Implementation of an AI Agent for Text-to-SQL Conversion in Relational Databases

AUTHOR

Dilshod Khudoyberdiev

PROGRAM

B.Sc. Information
Engineering

INSTITUTION

Hamburg University of
Applied Sciences (HAW
Hamburg)

SUPERVISORS

Prof. Dr. Björn Gottfried
Mr. Hans Schrader

DATE

May 13, 2026

SQL Is a Barrier Between People and Their Data

THE GAP



Non-technical staff can describe what they need in plain language



But they cannot write SQL or navigate database schemas



Result: dependency on data specialists for routine questions



Slows down decision-making, creates repetitive work for data teams

A TYPICAL QUESTION

"Which federal states had the highest ticket revenue last quarter?"

Requires: correct tables, joins, filters, aggregation, sorting

THE SOLUTION: **mxQueryChat**

A self-hosted Text-to-SQL application that translates natural language questions into executable SQL queries for relational databases.

Safe • Transparent • Improvable

Six Challenges Make Text-to-SQL Hard in Business Settings

01

Schema Understanding

System must use correct tables, columns, data types and relationships.

02

Schema Linking

"Revenue" might mean price × quantity or a stored column — mapping is error-prone.

03

Semantic Correctness

SQL can run but answer the wrong question (wrong join, filter, aggregation).

04

Value Grounding

Users refer to values they don't know the exact spelling of.

05

Robustness

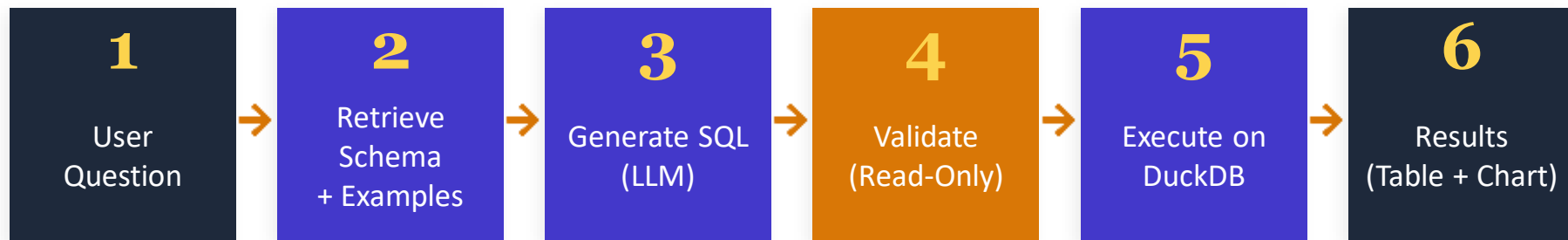
User questions vary; schemas evolve over time; small changes break behavior.

06

Safety & Trust

Generated SQL must never modify data. Users need to verify what runs.

The Agent Workflow: Retrieve → Generate → Validate → Execute



A very quick orientation: where each request goes (in order)

The LLM Is the Last Resort, Not the First — Four-Tier Pipeline

THE PROBLEM

Every user question triggers a 1–10s LLM call — even repeated questions, known patterns, and off-topic inputs. Unnecessary latency, cost, and hallucination risk.

THE FIX: FOUR-TIER GREEDY PIPELINE

T0	Guardrail Reject off-topic/write-intent	Instant
T1	Cache Hash table reuse of prior answers	$O(1)$
T2	Exact Match Normalized search in training CSV	$O(n)$
T3	Template Keyword-based decision tree	$O(1)$
T4	LLM Retry Schema-guided iterative refinement	$O(k)$

WHY THIS APPROACH

Greedy optimization: always try the cheapest algorithm first. Most questions are repeats or match known patterns — calling the LLM for these wastes time and adds risk.

THE ALGORITHM BEHIND EACH TIER

Session Cache	Dictionary ADT (Hashing)	$O(1)$
Exact Match	Linear Search + String Normalization	$O(n)$
Template	Rule-based Decision Tree	$O(1)$
LLM Retry	Iterative Refinement	$O(k), k \leq 3$

THE RESULT

Tiers 0–3 answer most questions in < 1ms with zero hallucination risk. The LLM is only called when no cheaper algorithm can handle the question.

RAG Grounds the LLM in Real Schema, Not Guesses

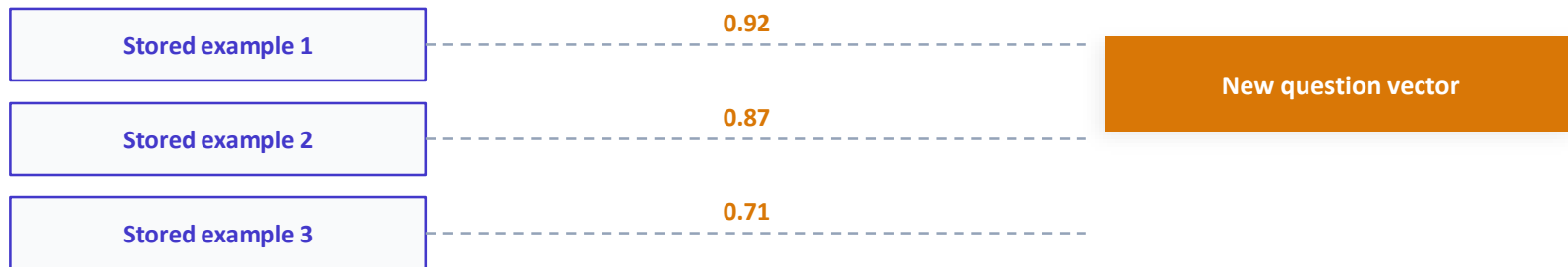
THE PROBLEM

Without schema context, the LLM invents table and column names (hallucinations). A wrong join path produces results that look correct but are semantically wrong — dangerous in business settings.

THE FIX: RETRIEVAL-AUGMENTED GENERATION

Before generation, retrieve relevant schema snippets and similar question→SQL examples from ChromaDB. Inject this context into the prompt so the LLM is constrained to real tables and columns.

WHY THIS WORKS: k-NEAREST NEIGHBOR SEARCH FINDS MEANING, NOT KEYWORDS



k-NN retrieval ($k=3$): find the 3 closest questions by cosine similarity — not by exact words.

THE RESULT

Grounded SQL replaces hallucinated SQL. Cross-language matching works: 'Welche Bundesländer hatten den höchsten Umsatz?' matches English training examples because the vectors are close.

ENGINEERING PRINCIPLE The LLM is only as good as the context it receives.

Safety by Design: Every Query Is Treated as Untrusted Input

THE PROBLEM

The LLM can generate any SQL — including DROP TABLE, DELETE, cross-joins that lock the database, or queries that stream gigabytes. One bad query can destroy the dataset.

WHY THIS APPROACH

Each check is $O(1)$ — it inspects fixed properties of the query string, not the data. Total overhead is $O(4) = O(1)$. Layers are independent: disabling one does not affect the others.

THE FIX: 4 INDEPENDENT $O(1)$ GUARDRAIL LAYERS

1

Read-Only SQL Guard

Blocks INSERT, UPDATE, DELETE, CREATE, DROP, TRUNCATE and 15+ unsafe keywords. Only SELECT/WITH pass.

2

Complexity Validator

Max JOINS (6), max CTEs (4), character limit (20,000). Blocks queries that are too expensive to run.

3

Row Limiter

Hard cap of 1,000 rows on every result set. Prevents streaming gigabytes of data to the UI.

4

Timeout Wrapper

15-second deadline. Query is killed if it exceeds the limit. Runs in a separate subprocess for isolation.

THE RESULT

T0 pre-filter rejects 41 out-of-scope patterns before any SQL work. 15/15 threats blocked. Zero writes reached the database.

Five Lines of Code, Five Algorithmic Decisions

PYTHON / app.py

```
# Tier 0 –  $O(1)$  guardrail: reject before any SQL work
local_block = get_local_guardrail_message(question)

# Tier 1 –  $O(1)$  hash lookup: session cache
cached_sql = st.session_state.sql_cache.get(question.lower())

# Tier 2 –  $O(n)$  linear search: exact training match
direct_sql = get_exact_training_sql(question, examples_df)

# Tier 3 –  $O(1)$  decision tree: template planner
template_sql, _ = build_template_sql(question)

# Tier 4 –  $O(k)$  iterative refinement: LLM with retry
sql, notes, error = generate_sql_with_retry(...)
```

← Decision 1

← Decision 2

← Decision 3

← Decision 4

← Decision 5

5 lines, 5 algorithms

Training Examples Are the Secret Weapon — Not the LLM

THE PROBLEM

Out-of-the-box LLM doesn't know domain terminology. "Umsatz" = revenue? "Tarifverbund" = tariff association? Without domain context, the LLM guesses wrong joins and grouping.

WHY NOT FINE-TUNING

Fine-tuning requires thousands of labeled examples, ML expertise, and GPU resources. Instead, we inject correct examples into the prompt at inference time — the LLM imitates the patterns it sees.

THE FIX: EMBED → RETRIEVE → INJECT PIPELINE



THE RESULT (CONTROLLED EXPERIMENT)

Without training examples: 0.25 execution accuracy (wrong joins, wrong grouping). With examples: 1.00 execution accuracy. The same 8 questions, the same LLM — only the k-NN retrieval context changed.

100% Accuracy on 20 In-Domain Questions — The MVP Works

PRODUCTION VALIDATION (20 REPRESENTATIVE DOMAIN QUESTIONS)

1.00

**EXECUTION
ACCURACY**

All queries returned
the correct result

1.00

**EXACT
MATCH**

Generated SQL matched
reference queries

1.00

**COMPILE
RATE**

All SQL syntactically
valid and executable

LATENCY (MEDIAN)

0.007s Total end-to-end
(generation + execution)

TEST COVERAGE

56

pytest tests passed
covering guardrails, execution,
retry logic, training, metrics

Without the four-tier pipeline, median latency would be 1–10s (LLM call). With it, most answers arrive in 0.007s.

ENGINEERING PRINCIPLE Measure everything. Numbers replace opinions.

15/15 Threats Blocked — Zero Writes Reached the Database

15/15

BLOCKED RATE

All unsafe test cases blocked

ZERO

WRITES EXECUTED

No data modification reached DB

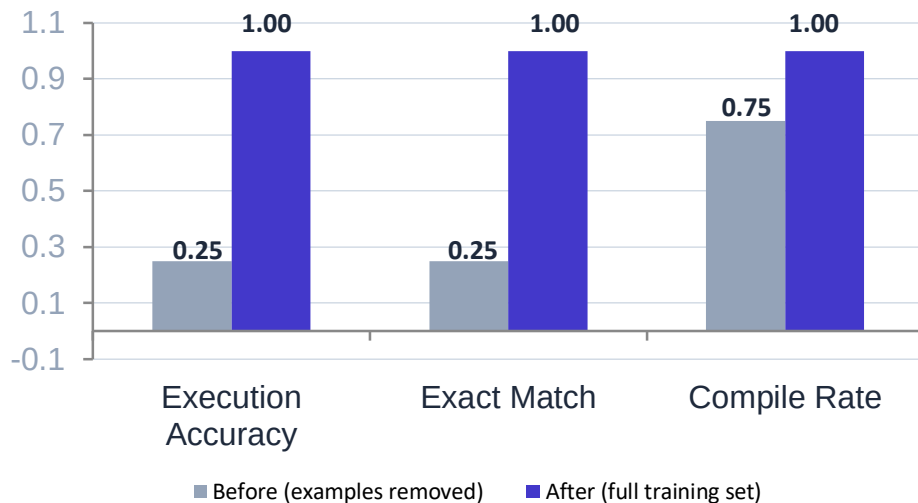
**WITHOUT THE
GUARDRAIL PIPELINE**

A single DROP TABLE from the LLM could destroy the dataset. The O(1) guard layers make this impossible.

Category	Cases	Blocked	Result
Write statements (INSERT, UPDATE, DELETE)	5	5	Blocked
DDL statements (CREATE, DROP)	2	2	Blocked
Multi-statement SQL	3	3	Blocked
Complexity-limit cases	3	3	Blocked
Off-topic / invalid inputs	2	2	Blocked

Controlled Experiment Proves: Vector Retrieval Is Load-Bearing

Controlled experiment: same 8 harder questions, tested twice. First run: all training examples removed from ChromaDB — k-NN had nothing to retrieve. Second run: full training set restored. Result: execution accuracy 0.25 → 1.00.



KEY FINDING

0.25 → 1.00

Execution Accuracy
after restoring k-NN retrieval

WHAT THIS MEANS ALGORITHMICALLY

Without k-NN retrieval: LLM generates wrong joins and grouping.

With retrieval: LLM receives correct SQL patterns and replicates them.

This proves that k-NN retrieval — not the LLM — is the load-bearing mechanism for correctness.

The Boundary of the System — Held-Out Benchmark and Honest Limitations

HELD-OUT BENCHMARK (10 QUESTIONS)

Execution Accuracy **0.00**

Exact Match **0.00**

Compile Rate **0.90**

90% compiled — but without training support, semantic correctness was not achieved.

THE ACHIEVEMENT

- ✓ 100% on supported business-style questions
- ✓ Strict safety: zero writes in all tested scenarios
- ✓ Domain adaptation through editable examples
- ✓ Sub-second response times for interactive use

SEMANTIC ERROR EXAMPLES

"Revenue by tariff and state"

→ Added unnecessary year grouping

"Top 3 ticket types per state"

→ Dropped ticket-type dimension

"Revenue by state and ticket for 2025"

→ Grouped only by state

THE BOUNDARY

- △ Strong dependence on stored training examples
- △ No semantic verification beyond compile checks
- △ No clarification for ambiguous questions
- △ Single DuckDB database — no multi-DB support

Three Engineering Contributions, One Working MVP



1. THE PIPELINE

- ▶ Dictionary ADT $O(1)$ → Linear Search $O(n)$ → Decision Tree $O(1)$ → Iterative Refinement $O(k \leq 3)$
- ▶ RAG retrieval via ChromaDB k-NN for schema grounding
- ▶ Three-attempt iterative refinement with error feedback



2. THE SAFETY LAYER

- ▶ Set membership check: 20-keyword $O(1)$ lookup before every query
- ▶ Execution policy: timeout, row cap, complexity limits
- ▶ 15/15 threats blocked, zero writes confirmed



3. THE TRAINING LOOP

- ▶ k-NN (cosine similarity, $k=3$) retrieves closest examples — not keywords, vectors
- ▶ Accuracy improved from 0.25 → 1.00 (controlled experiment)
- ▶ Domain adaptation without code changes

CENTRAL FINDING

A self-hosted Text-to-SQL system reliably serves non-technical users by combining k-NN vector retrieval, a greedy $O(1) \rightarrow O(n) \rightarrow O(1) \rightarrow O(k)$ pipeline, and an $O(1)$ guardrail layer. 100% accuracy on supported patterns, sub-second latency.

The System in Practice: SQL Is Always Shown Before It Runs

mxQueryChat

← → ↻ localhost:8501

mxQueryChat

Choose page

☒ New Question

☐ Training Data

☐ Chat History

Session Metrics

Questions 6

SQL Generated 4

Generation Failed 2

Blocked Requests 2

Query Success 2

Query Failed 0

Data Source (DuckDB)

mxquerychat.duckdb

Edits: yes | Size: ~12.1k

Reload Dataset

Refresh Schema

► Schema Tree

mxquerychat

Ask questions in plain language, get SQL query and results. Built with Vanna agent and DuckDB.

New Chat

Step 1: Ask a data question

Example 1

Example 2

Example 3

Example 4

Which 5 federal states generated the most ticket revenue in 2025?

Send

i Loaded from session cache.

Step 2: Review SQL

```
SELECT tv.jahr, rb.bundesland_name, SUM(tv.umsatz_eur) AS umsatz_eur
FROM ticket_verkaeufe tv
JOIN postleitzahlen p ON CAST(tv.plz AS VARCHAR) = p.plz
JOIN regionen_bundesland rb ON p.bundesland_code2 = rb.bundesland_code2
WHERE tv.jahr = 2025 GROUP BY tv.jahr, rb.bundesland_name ORDER BY umsatz_eur DESC
```

Run Query

✓ Read-only check passed.

Step 4: Results · Rows: 5 | Time: 0.009s

jahr	bundesland_name	umsatz_eur
2025	Hamburg	3,847,213
2025	Bayern	3,612,584
2025	Nordrhein-Westfalen	3,401,156
2025	Baden-Württemberg	3,287,910
2025	Berlin	3,145,802

From MVP to Production: Five Priorities for Future Work

1

Semantic Verification

Check expected result dimensions, grouping columns, and join paths before execution.

2

Clarification Handling

Detect ambiguous questions and ask follow-up instead of guessing.

3

Training Expansion

Systematic example management to increase supported query coverage.

4

Async Training

Non-blocking training updates with progress feedback.

5

Multi-Database Support

Extend beyond DuckDB with additional connectors.

THANK YOU

I Welcome Your Questions — Let's Have a Demo!

Dilshod Khudoyberdiev • B.Sc. Information Engineering • HAW Hamburg • May 2026